1. Parallel Adder:

Program

```
module adder4 ( carryin,x,y,sum,carryout);
input carryin;
input [3:0] x,y;
output [4:0] sum;
output carryout;
fulladd stage0 (carryin,x[0],y[0],sum[0],c1);
fulladd stage1 (c1,x[1],y[1],sum[1],c2);
fulladd stage2 (c2,x[2],y[2],sum[2],c3);
fulladd stage3 (c3,x[3],y[3],sum[3],carryout);
assign  sum[4]=carryout;
endmodule
```

Testbench:

```
module adder4_t ;
reg [3:0] x,y;
reg carryin;
wire [4:0] sum;
wire carryout;
adder4 a1 (carryin,x,y,sum,carryout);
initial
begin
x = 4'b0000; y= 4'b0000;carryin = 1'b0;
#20 x =4'b1111; y = 4'b1010;
#40 x =4'b1011; y =4'b0110;
#40 x =4'b1111; y=4'b1111;
#50 $finish;
end
endmodule

module fulladd (cin,x,y,s,cout);
input cin,x,y;
output s,cout;
assign s = x^y^cin;
assign cout =( x & y) | (x & cin) |( y & cin);
endmodule
```

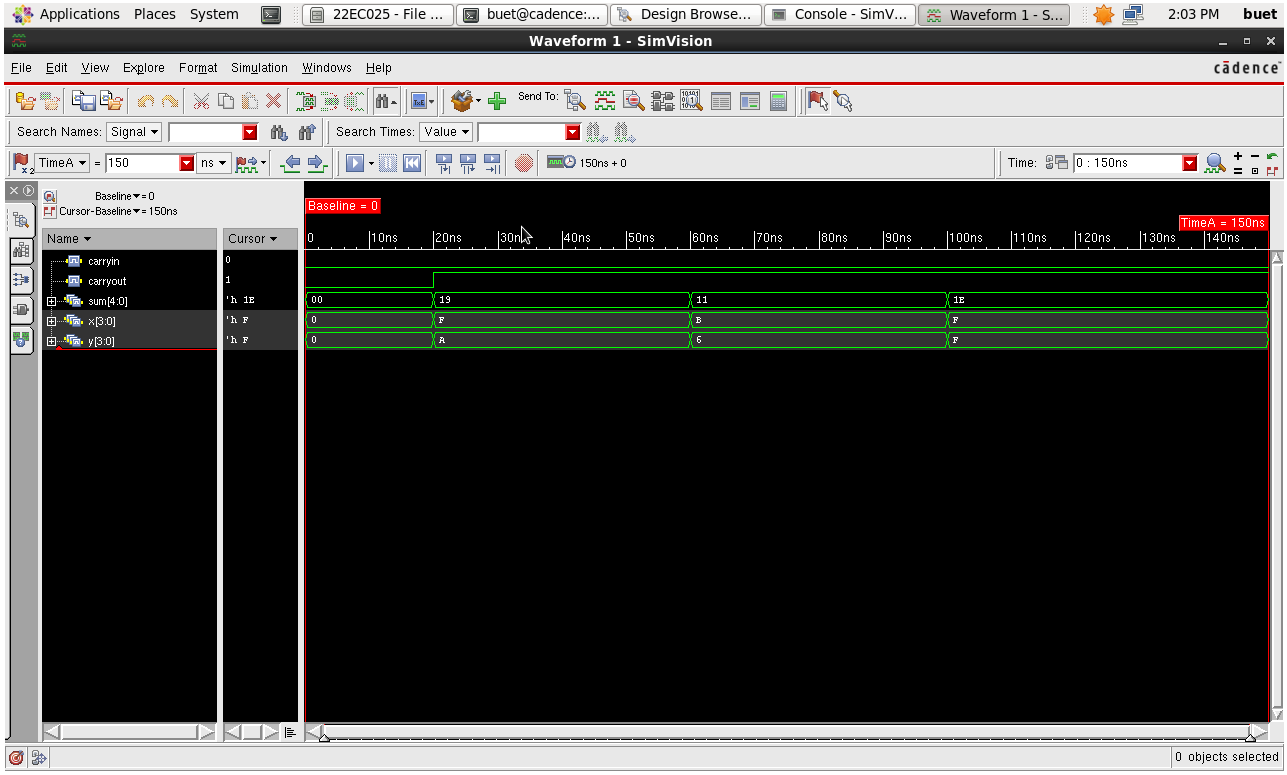

Fig.1 : Waveform of Parallel Adder

Synthesize the design using Constraints and analyse reports, critical path and Max Operating Frequency.

Step 1: Getting Started

Make sure you close out all the Incisive tool windows first.

Synthesis requires three files as follows, Liberty Files (.lib)

Verilog/VHDL Files (.v or .vhdl or .vhd)

SDC (Synopsis Design Constraint) File (.sdc)

Step 2: Creating an SDC File

In your terminal type “gedit constraints_top.sdc” to create an SDC File if you do not have one.

The SDC File must contain the following commands

```
set_input_delay -max 0.8 [get_ports "A"]
set_input_delay -max 0.8 [get_ports "B"]
set_input_delay -max 0.8 [get_ports "C0"]
set_output_delay -max 0.8 [get_ports "S"]
set_output_delay -max 0.8 [get_ports "C4"]
```

Step 3 : Performing Synthesis

The Liberty files are present in the below path,
/home/install/FOUNDRY/digital/<Technology_Node_number>nm/dig/lib/
The Available technology nodes are 180nm ,90nm and 45nm.

In the terminal, initialise the tools with the following commands if a new terminal is being used.
csh source /home/install/cshrc

The tool used for Synthesis is “Genus”. Hence, type “**genus -gui**” to open the tool

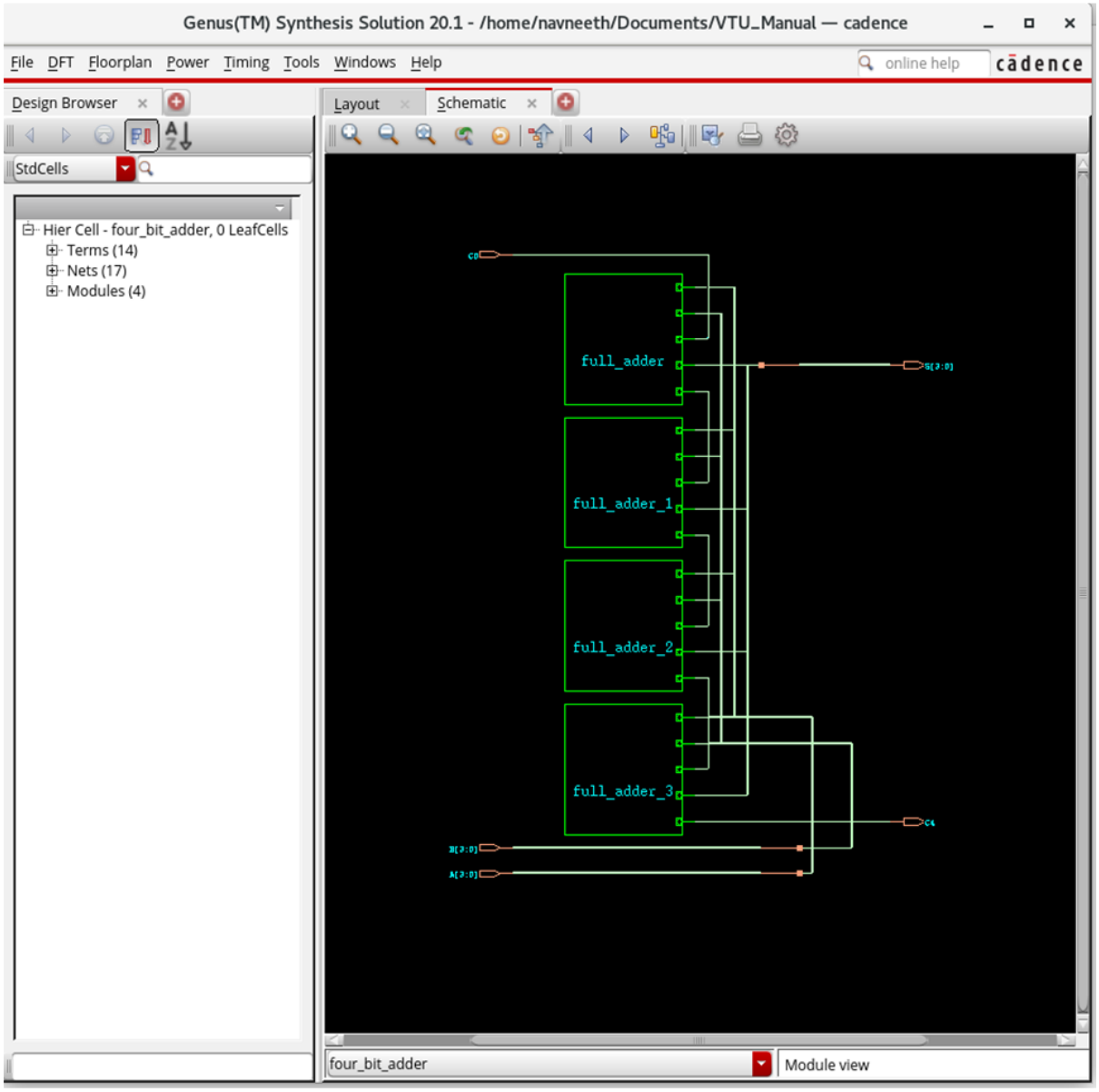

Fig.1.2 : Schematic Capture of 4bit Adder

2. 4-bit Shift & Add Multiplier

```
module shift_add_multiplier(clk,reset,A,B,P);
   input clk,reset;
   input [3:0]A,B;     // Multiplicand and Multiplier
   output reg [7:0]P; // Product
reg [7:0]multiplicand;
reg [3:0]multiplier;
reg [2:0]count;
always @(posedge clk or posedge reset)
begin
   if (reset)
   begin
      multiplicand <= 0;
      multiplier   <= 0;
      P           <= 0;
      count       <= 0;
   end
   else
   begin
      if (count == 0)
      begin
         multiplicand <= {4'b0000, A}; // Extend A to 8 bits
         multiplier   <= B;
         P           <= 0;
         count       <= 4;
      end
      else
      begin
         if (multiplier[0] == 1)
            P <= P + multiplicand;

         multiplicand <= multiplicand << 1;
         multiplier   <= multiplier >> 1;
         count       <= count - 1;
      end
   end
end
endmodule
module tb_multiplier;
reg clk,reset;
reg [3:0]A,B;
wire [7:0]P;

shift_add_multiplier uut(clk,reset,A,B,P);
initial
clk = 0;
always #5 clk = ~clk;
initial
begin
   reset = 1;
   #10 reset = 0;
   A = 4'b0101; // 5
   B = 4'b0011; // 3
#50
  A = 4'b1011; // 11
  B = 4'b0111; // 7
```

```
    #50 $finish;
end
endmodule
```

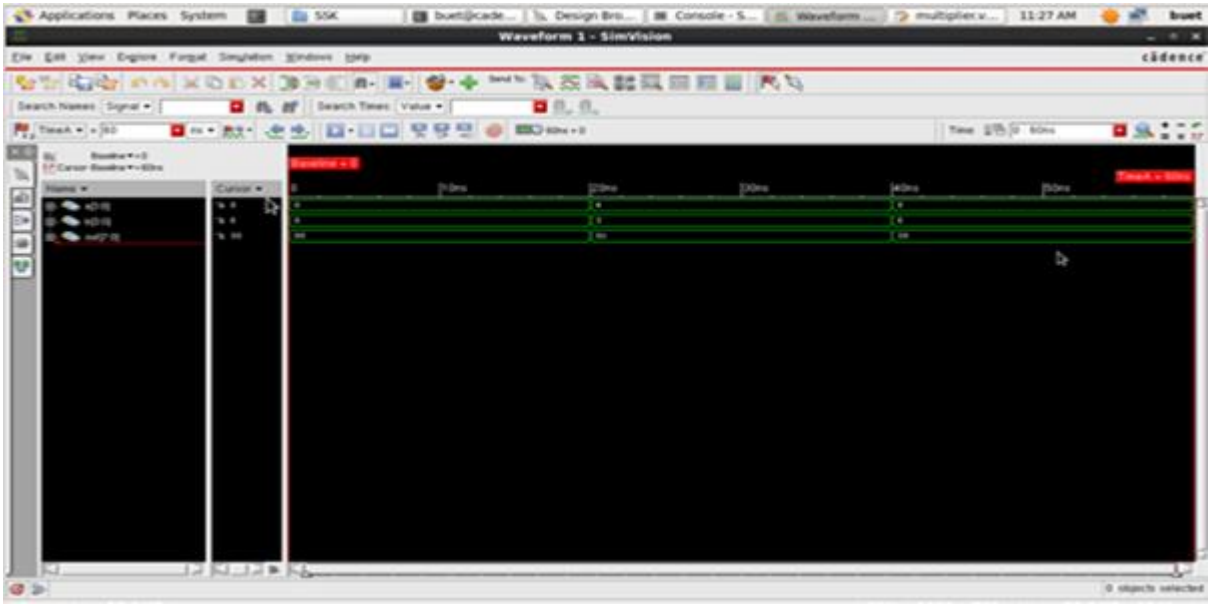

Fig:2. Waveform of shift & add multiplier

3. a. 32-bit ALU using case statement

```
module alu_32bit_case(y,a,b,f);
input [31:0]a;
input [31:0]b;
input [2:0]f;
output reg [31:0]y;
always@(*)
begin
case(f)
3'b000:y=a&b; //AND Operation
3'b001:y=a|b; //OR Operation
3'b010:y=~(a&b); //NAND Operation
3'b011:y=~(a|b); //NOR Operation
3'b100:y=a+b; //Addition
3'b101:y=a-b; //Subtraction
3'b110:y=a*b; //Multiply
default:y=32'bx;
endcase
end
endmodule
//Test Bench//
module alu_32bit_tb_case;
reg [31:0]a;
reg [31:0]b;
reg [2:0]f;
wire [31:0]y;
alu_32bit_case test2 (y,a,b,f);
initial
begin
a=32'h00000000;
b=32'hFFFFFFFF;
#10 f=3'b000;
#10 f=3'b001;
#10 f=3'b010;
#10 f=3'b100;
#10 f=3'b101;
#10 f=3'b110;
#10 f=3'b111;
#50 $finish;
end
endmodule
```

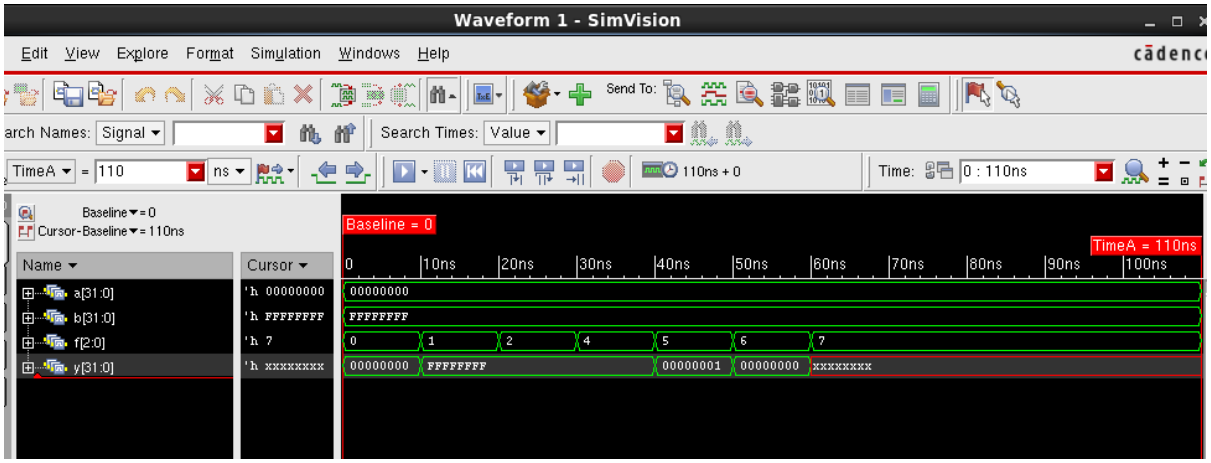

Fig.3a.: Waveform of ALU

3. b. 32-bit ALU using if else statement

```
module alu_32bit_if(y,a,b,f);
input [31:0]a;
input [31:0]b;
input [2:0]f;
output reg [31:0]y;
always@(*)
begin
if(f==3'b000)
y=a&b; //AND Operation
else if (f==3'b001)
y=a|b; //OR Operation
else if (f==3'b010)
y=a+b; //Addition
else if (f==3'b011)
y=a-b; //Subtraction
else if (f==3'b100)
y=a*b; //Multiply
else
y=32'bx;
end
endmodule
//Test bench //
module alu_32bit_tb_if;
reg [31:0]a;
reg [31:0]b;
reg [2:0]f;
wire [31:0]y;
alu_32bit_if test(y,a,b,f);
initial
begin
a=32'h00000000;
b=32'hFFFFFFFF;
#10 f=3'b000;
#10 f=3'b001;
#10 f=3'b010;
#10 f=3'b100;
#10 f=3'b101;
#50 $finish;
end
endmodule
```

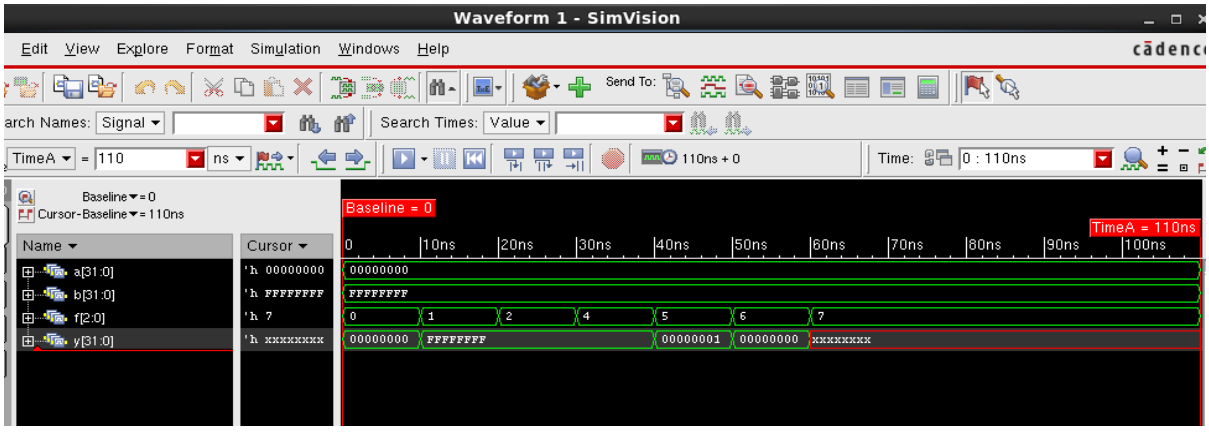

Fig.3b.: Waveform of ALU

4.Verilog code for flip flops

D flip flop

```
module dff1(q,qb,clk,rst,din);
input clk,rst,din;
output reg q;
output qb;
always @(posedge clk or posedge rst)
begin
if(rst)
q<=1'b0;
else
q<=din;
end
assign qb= ~q;
endmodule
//test bench//
module dff_t;
reg clk,din,rst;
wire q,qb;
dff1 U1(q,qb,clk,rst,din);
initial
clk=1'b0;
always
#10 clk=~clk;
initial
begin
din=1'b0;
rst=1'b1;
#20 rst=1'b0'
#10 din=1'b1;
#15  rst=1'b1;
#18 din=1'b0
#1 din=1'b1;
#20 din=1'b0;
#10 $finish;
end
endmodule
```

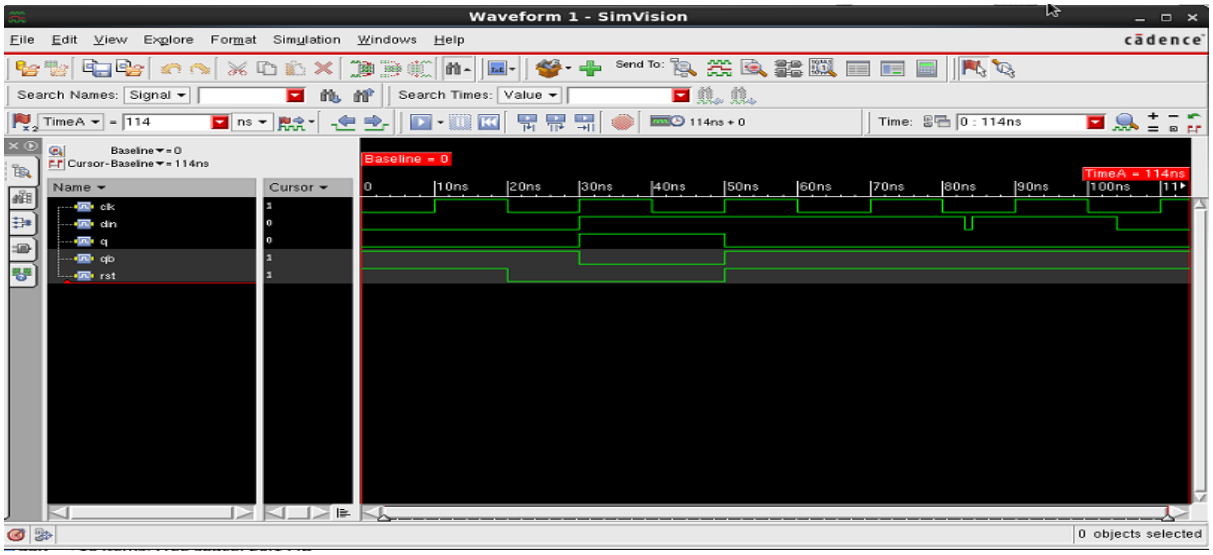

Fig.4a.: Waveform of D-Flipflop

JK flip flop

```
module jk_ff(q,qbar,clk,rst,j,k);
input clk,rst,j,k;
output q,qbar;
reg q,tq;
always@(posedge clk or negedge rst)
begin
if(!rst)
begin
q<=1'b0;
tq<=1'b0;
end
else
begin
if(j==1'b1 && k==1'b0)
q<=j;
else if(j==1'b0 && k==1'b1)
q<=1'b0;
else if(j==1'b1 && k==1'b1);
begin
tq<=~tq;
q<=tq;
end
end
end
assign qbar=~q;
endmodule
//Testbench//
module jk_ff_test;
reg clk,rst,j,k;
wire q,qbar;
jk_ff inst(q,qbar,clk,rst,j,k);
initial
clk=1'b0;
always
#10 clk=~clk;
initial
begin
j=1'b0;k=1'b0;rst=1'b0;
#30 rst=1'b1;
#60 k=1'b1;
#29 j=1'b1;
#15 k=1'b1;
#20 j=1'b0;
#40 j=1'b1;
#25 k=1'b0;
#5 j=1'b0;
#50 rst=1'b0;
#10 $finish;
end
endmodule
```

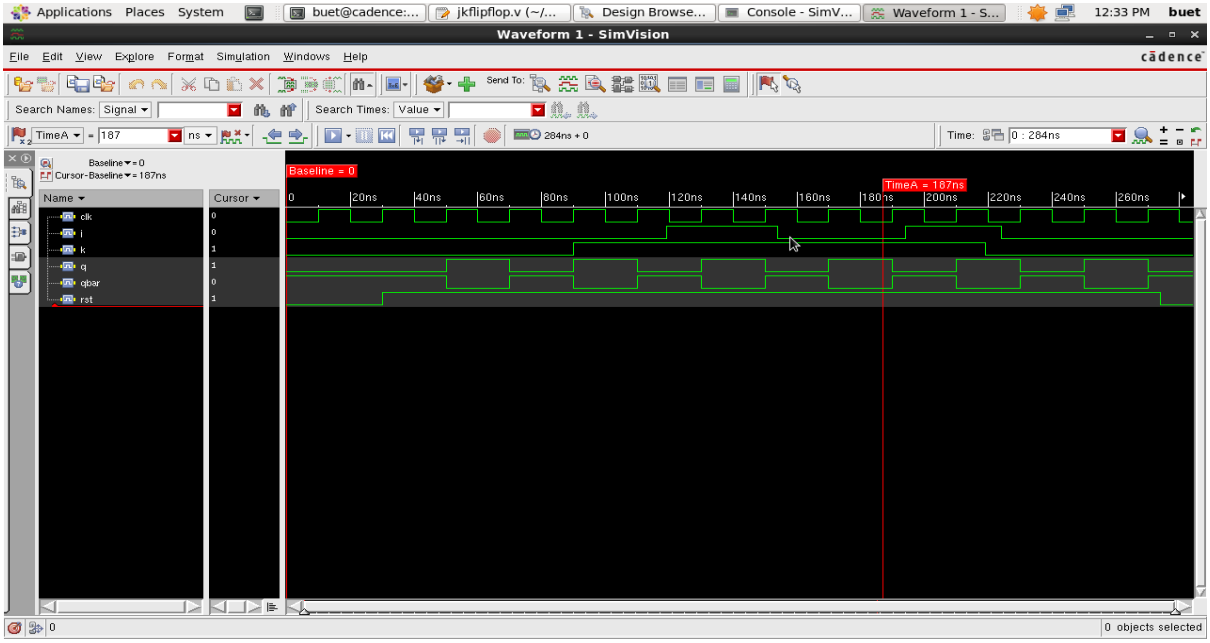

Fig.4b.: Waveform of JK-Flipflop

SR flip flop

```
module SR_ff(q,qbar,s,r,clk,rst);
output q,qbar;
input clk,s,r,rst;
reg tq;
always @(posedge clk or rst)
begin
if(s==1'b0&&r==1'b0)
tq<=tq;
else if(s==1'b0&&r==1'b1)
tq<=1'b0;
else if(s==1'b1&&r==1'b0)
tq<=1'b1;
else if(s==1'b1&&r==1'b1)
tq<=1'bx;
end
assign q=tq;
assign qbar=~tq;
endmodule
//Test bench//
module SR_ff_t;
reg clk,s,r,rst;
wire q,qbar;
SR_ff ff1(q,qbar,s,r,clk);
initial
clk=1'b0;
always
#10 clk=~clk;
initial
begin
s=1'b0;r=1'b0;rst=1'b0;
#30 s=1'b1;
#29 s=1'b0;
```

```
#1 r=1'b1;rst=1'b1;
#30 s=1'b1;
#15 rst=1'b0;
#30 r=1'b0;
#20 s=1'b0;
#19 s=1'b1;
#200 s=1'b1;r=1'b1;
#50 s=1'b0;r=1'b0;
#50 s=1'b1;r=1'b0;
#10 $finish;
end
endmodule
```

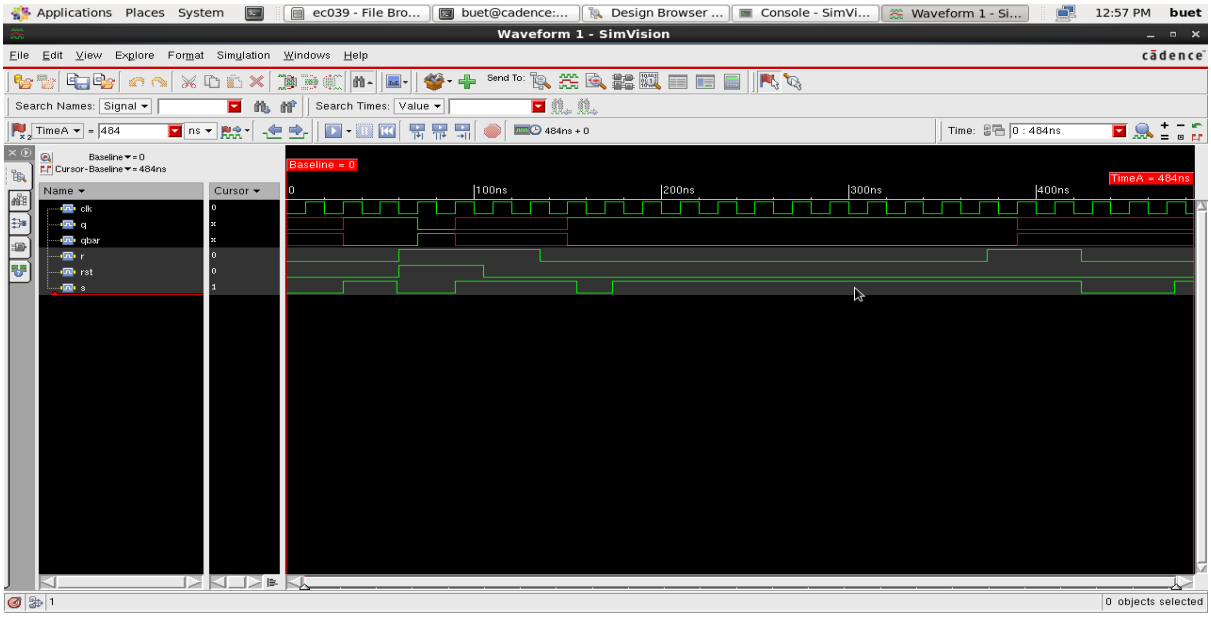

Fig.4c.: Waveform of SR-Flipflop

5a. Verilog code for synchronous counter

```
module counter(clk,rst,m,count);
input clk,rst,m;
output reg [3:0]count;
always @(posedge clk or negedge rst)
begin
if(!rst)
count=0;
else if(m)
count=count+1;
else
count=count-1;
end
endmodule
//test bench//
module counter_test;
reg clk,rst,m;
wire [3:0]count;
counter counter1(clk,rst,m,count);
initial
clk=0;
always
#5 clk= ~clk;
initial
begin
rst=0; m=0; // Condition for Up-Count
#25
rst=1;
 m=1;
#200 m=0; // Condition for Down-Count
#25 rst=1;
#200 m=1;
#140 m=0;
#250 $finish; // Finishing Simulation at t=140ns
end
endmodule
```

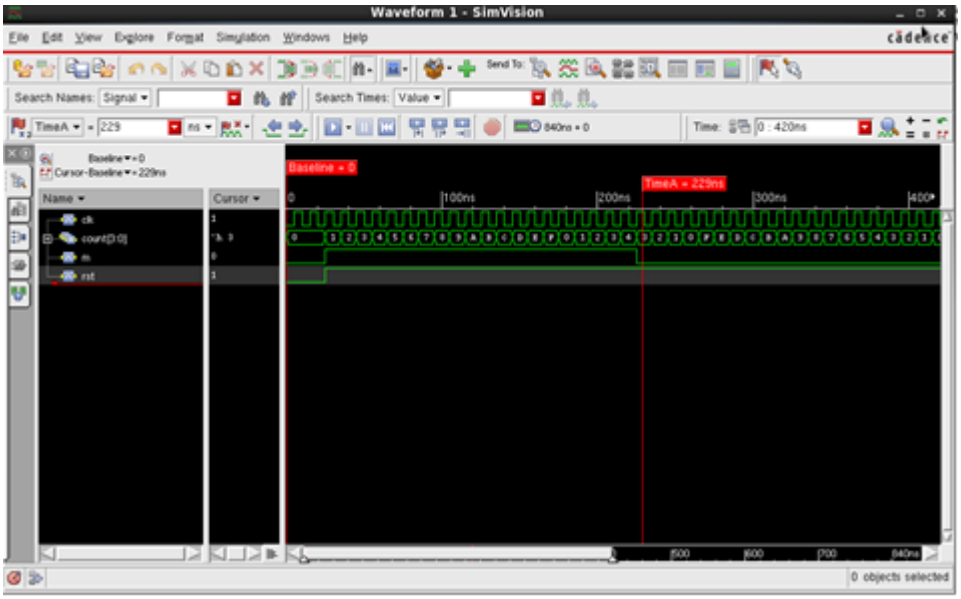

Fig: waveform of synchronous counter

5.b Asynchronous counter

```
module ripple_counter(clock,toggle,reset,count);
 input clock,toggle,reset;
 output [3:0]count;
 reg [3:0]count;
 wire c0,c1,c2;
 assign c0 = count[0],c1 = count[1],c2 = count[2];

  always @ (posedge reset or posedge clock)
    if (reset == 1'b1) count[0] <= 1'b0;
   else if (toggle == 1'b1) count[0] <= ~count[0];

 always @ (posedge reset or negedge c0)
    if (reset == 1'b1) count[1] <= 1'b0;
    else if (toggle == 1'b1) count[1] <= ~count[1];

 always @ (posedge reset or negedge c1)
    if (reset == 1'b1) count[2] <= 1'b0;
  else if (toggle == 1'b1) count[2] <= ~count[2];

 always @ (posedge reset or negedge c2)
    if (reset == 1'b1) count[3] <= 1'b0;
    else if (toggle == 1'b1) count[3] <= ~count[3];
endmodule
        //Test bench structure
module ripple_counter_t ;
reg clock,toggle,reset;
wire [3:0] count ;
ripple_counter r1 (clock,toggle,reset,count);
initial
   clock = 1'b0;
always
   #5 clock = ~clock;
initial
begin
  reset = 1'b0; toggle = 1'b0;
  #10 reset = 1'b1; toggle = 1'b1;
  #10 reset = 1'b0;
  #190 reset = 1'b1;
  #20 reset = 1'b0;
  #100 reset = 1'b1;
  #40 reset = 1'b0;
  #25 $finish;
end
endmodule
```

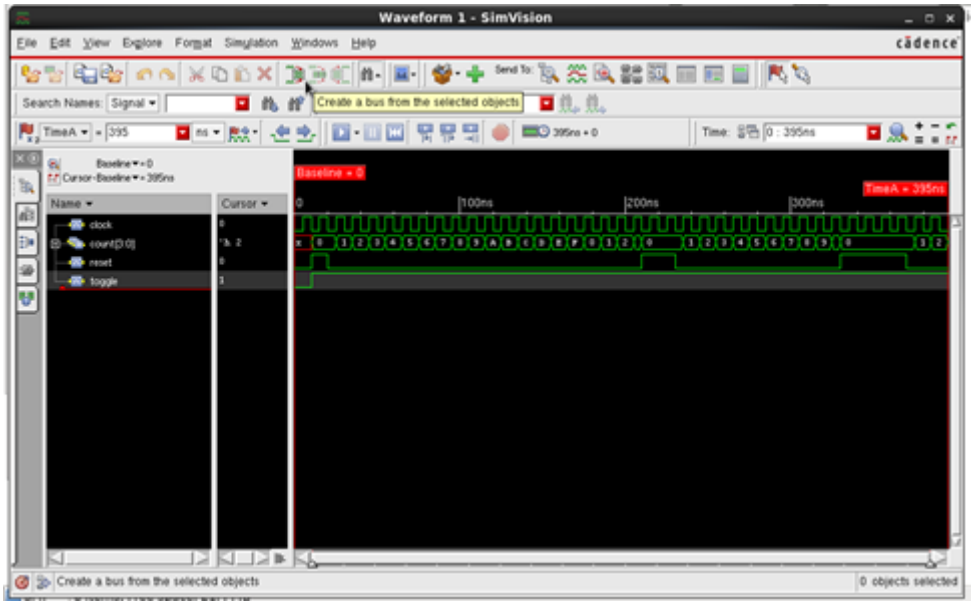

Fig: waveform of asynchronous counter

6. Inverter

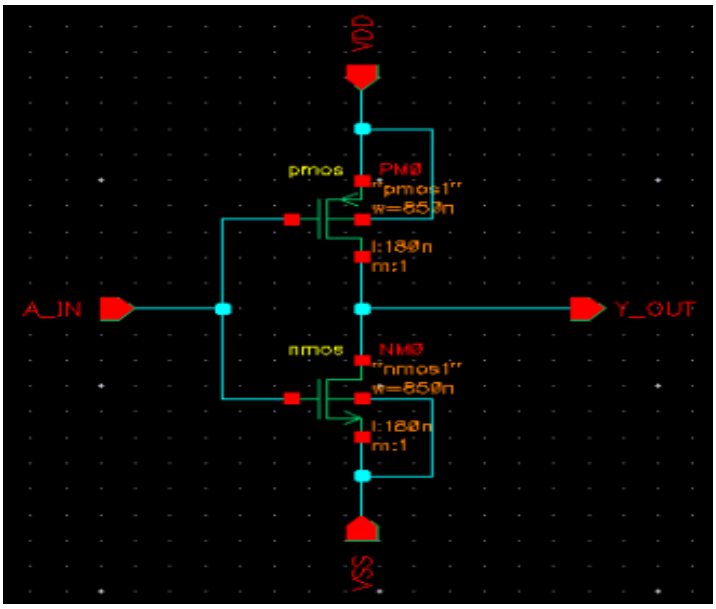

Fig. 6;1: Schematic of Inverter Wn=Wp

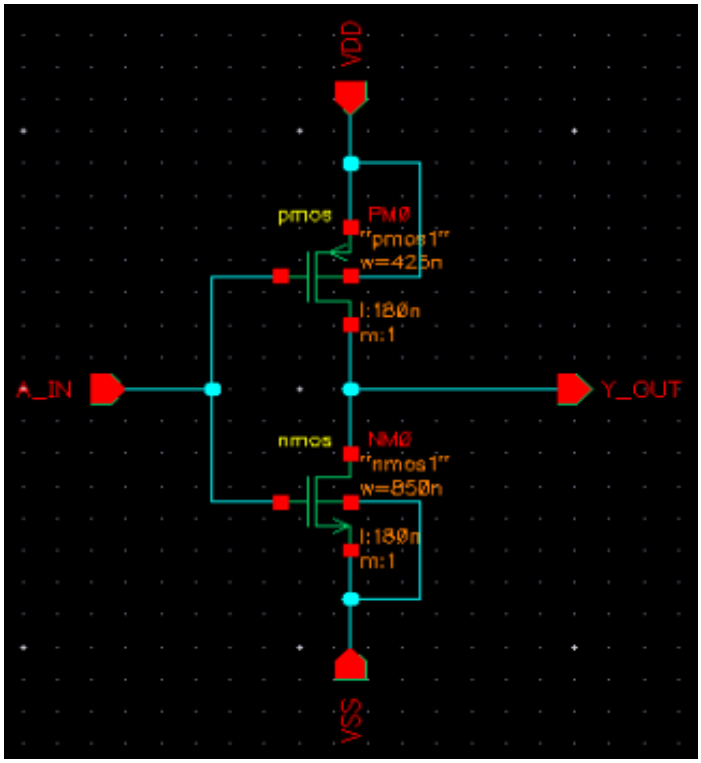

Fig. 6;2: Schematic of Inverter Wp=Wn/2

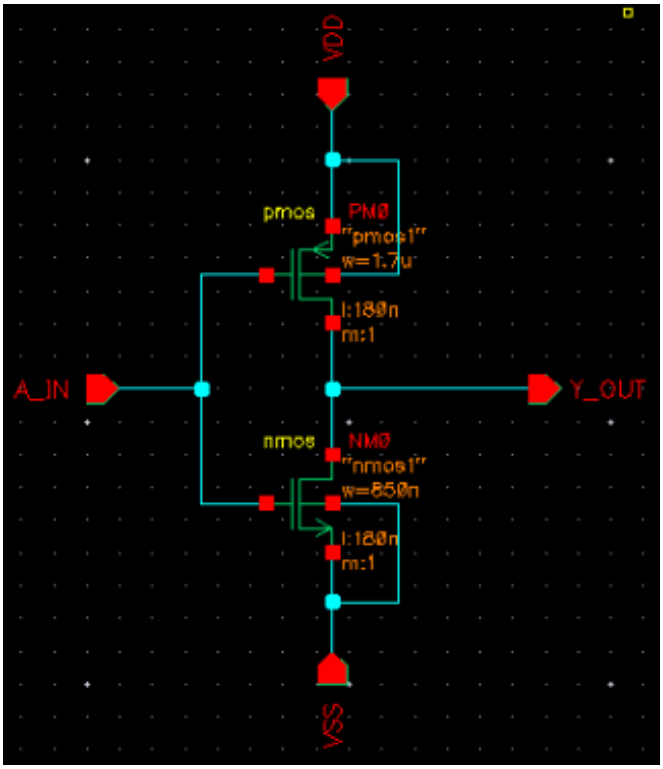

Fig. 6;3: Schematic of Inverter Wp=2Wn

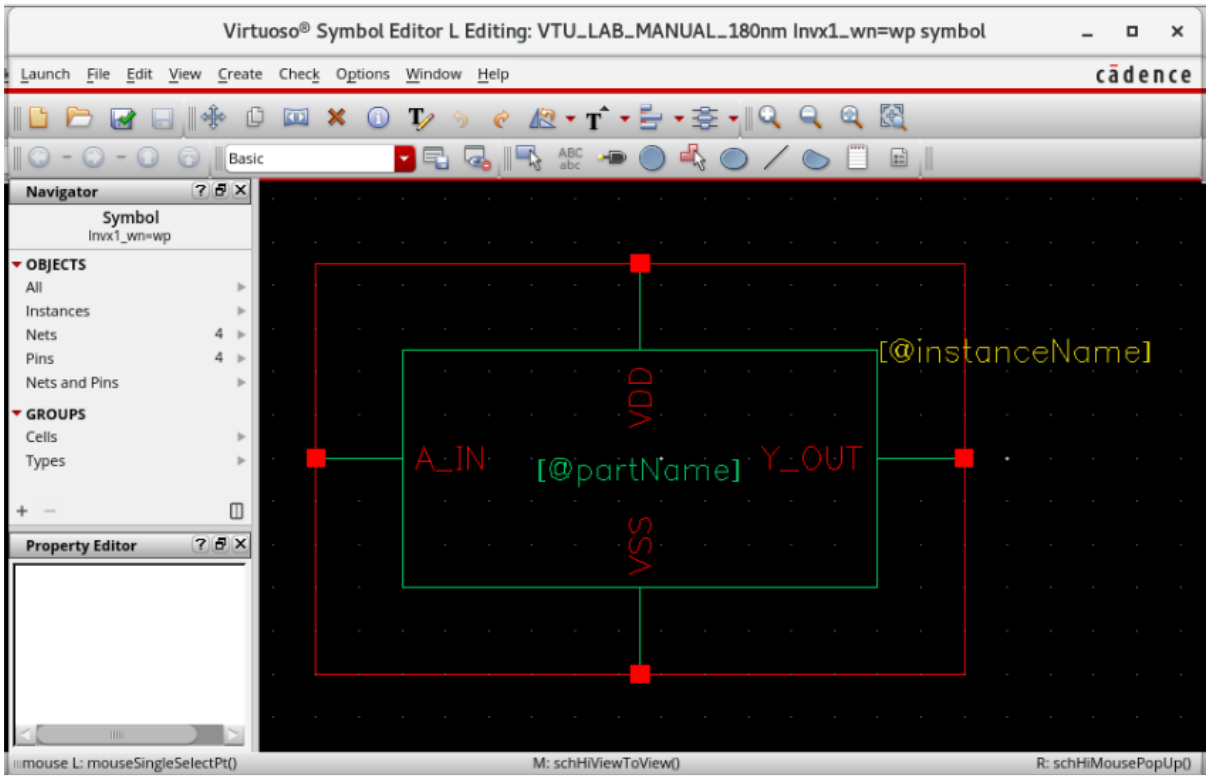

Fig. 6;4: Symbol of Inverter

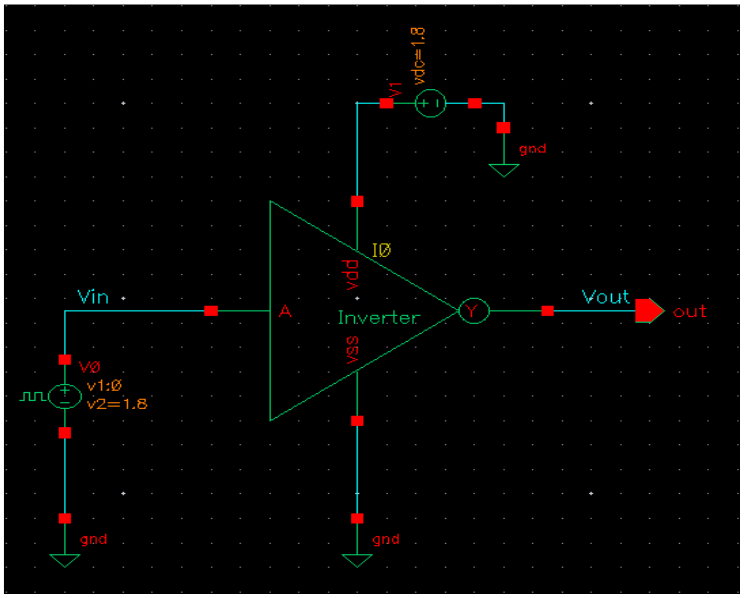

Fig. 6;5: Test generation of Inverter

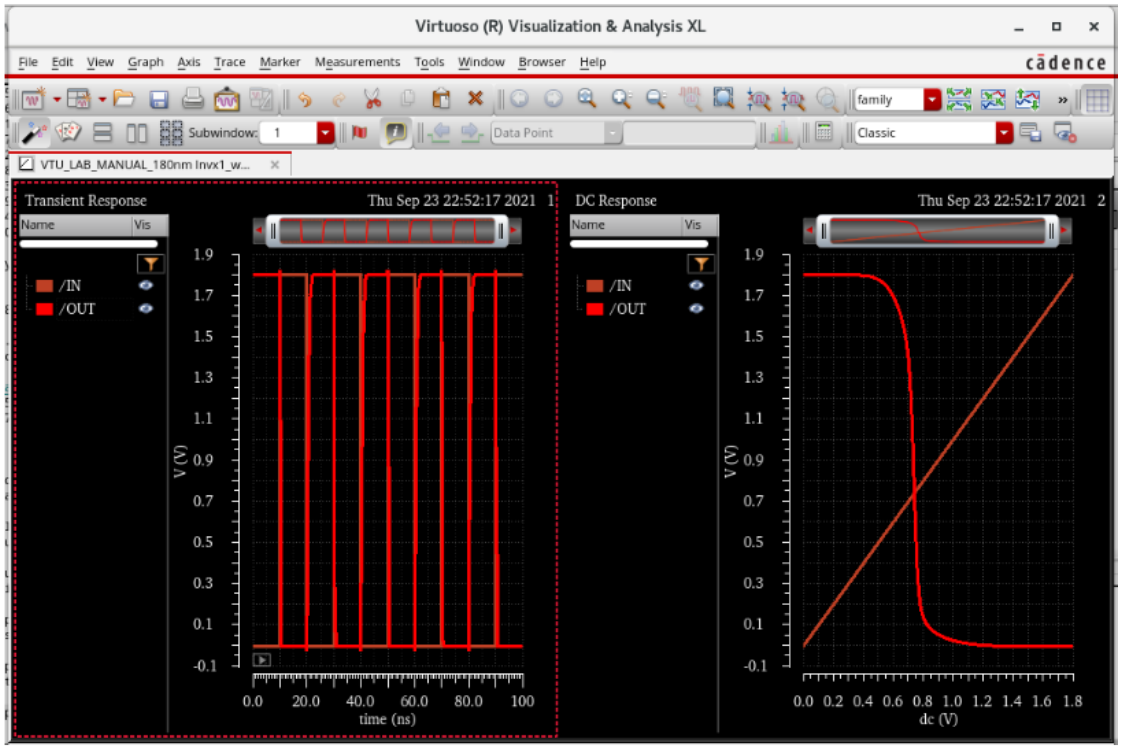

Fig. 6;6: Simulated waveforms of Inverter

Table –1 : Values of tp_{HL}, tp_{LH} and t_{PD} for different geometries

Width Settings	MOSFET	Width	tpLH	tpHL	tpd
Wp =Wn	PMOS				
	NMOS				
Wn =2Wp	PMOS				
	NMOS				
Wn = Wp/2	PMOS				
	NMOS				

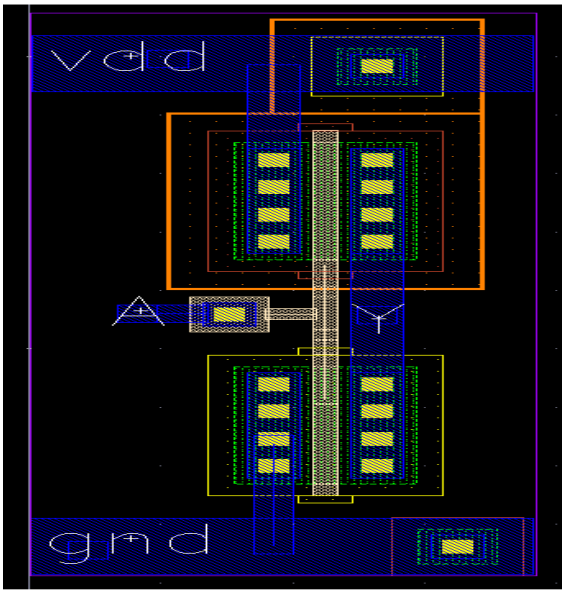

Fig. 6;7: Layout of Inverter

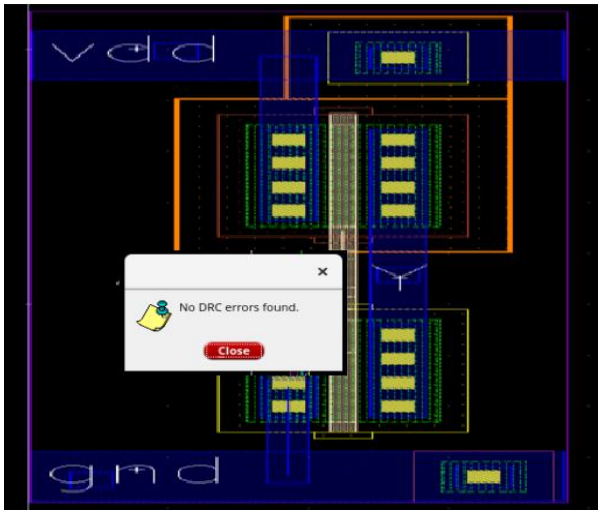

Fig.6.8: Result of DRC Check if the layout is DRC clean

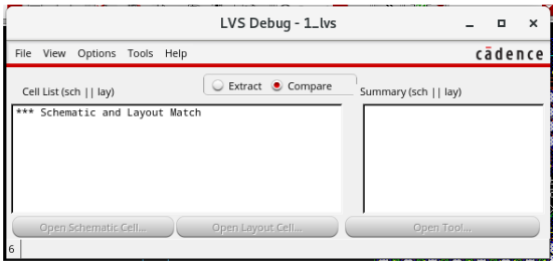

Fig.6.9: "LVS Debug" window

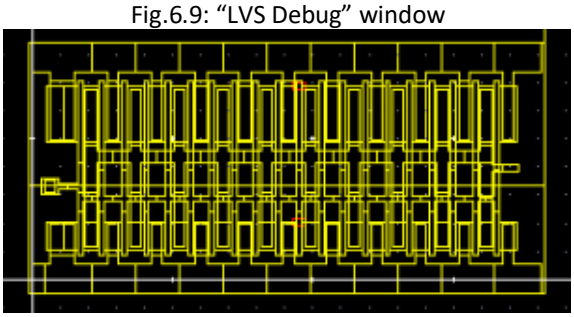
Fig.6.10: Extracted View

Table – 2: Values of tp_{HL}, tp_{LH} and t_{PD} for CMOS Inverter with $W_P/W_N=4/2$

MOSFET	Length	Width	tpLH	tpHL	tpd
PMOS	180n				
NMOS	180n				

7. Two Input CMOS NOR Gate

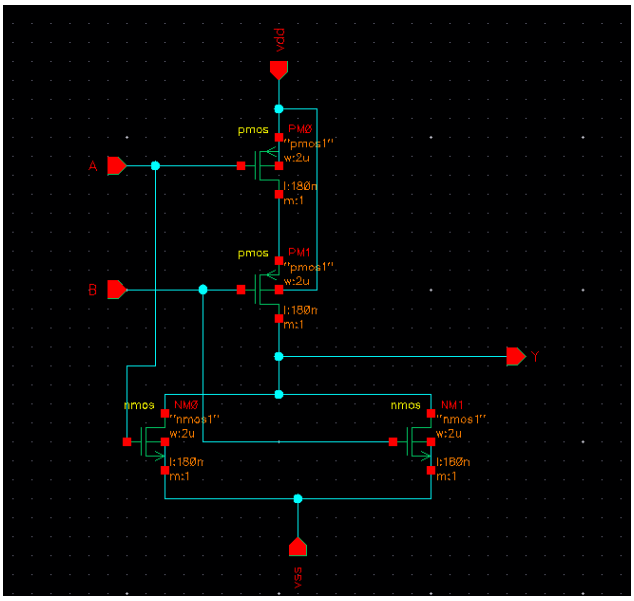

Fig.7.1: Schematic of NOR Gate

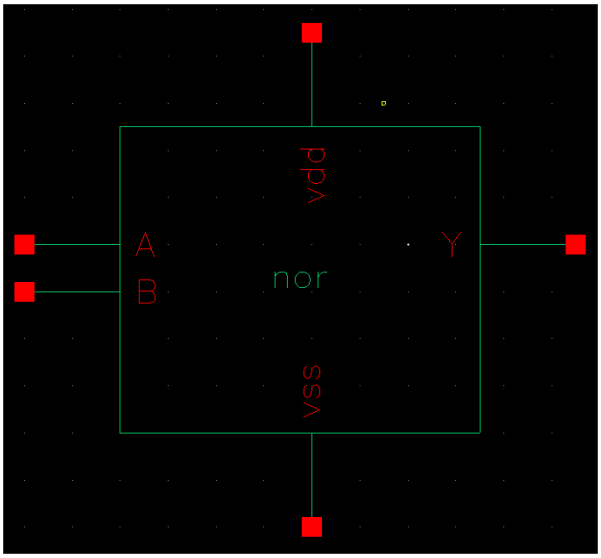

Fig.7.2: Symbol of NOR Gate

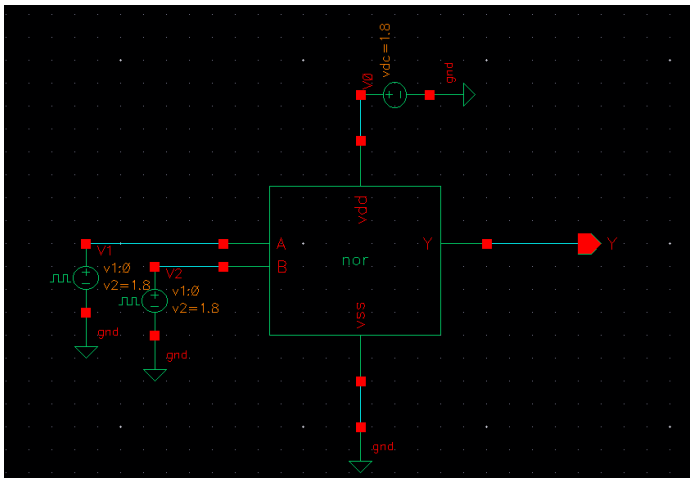

Fig.7.3: Test generation of NOR Gate

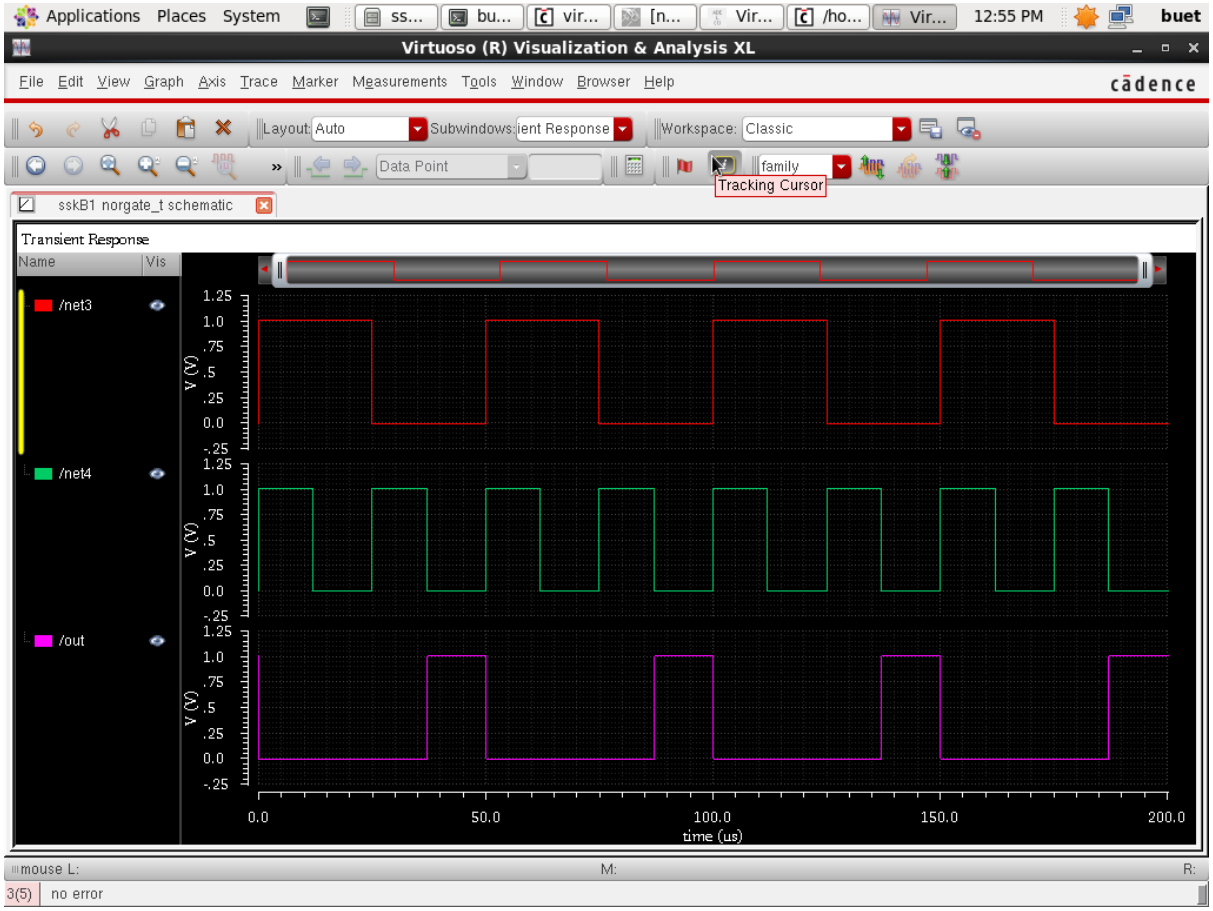

Fig.7.4: Waveforms of NOR Gate

Delay Elements for 2 – input CMOS NOR Gate (Prelayout Simulation)

MOSFET	Length	Width	tpLH	tpHL	tpd
PMOS	180n				
NMOS	180n				

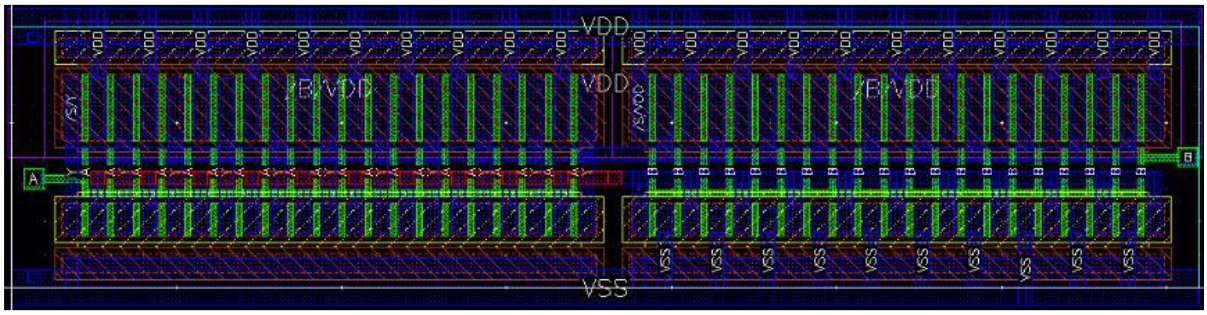

Fig.7.5: Layout of NOR Gate

Delay Elements for 2 – input CMOS NOR Gate (Post layout Simulation)

MOSFET	Length	Width	tpLH	tpHL	tpd
PMOS	180n				
NMOS	180n				

8. Construct the schematic using Boolean Expression using CMOS-Logic $Y = \overline{(B+CD+E)}$

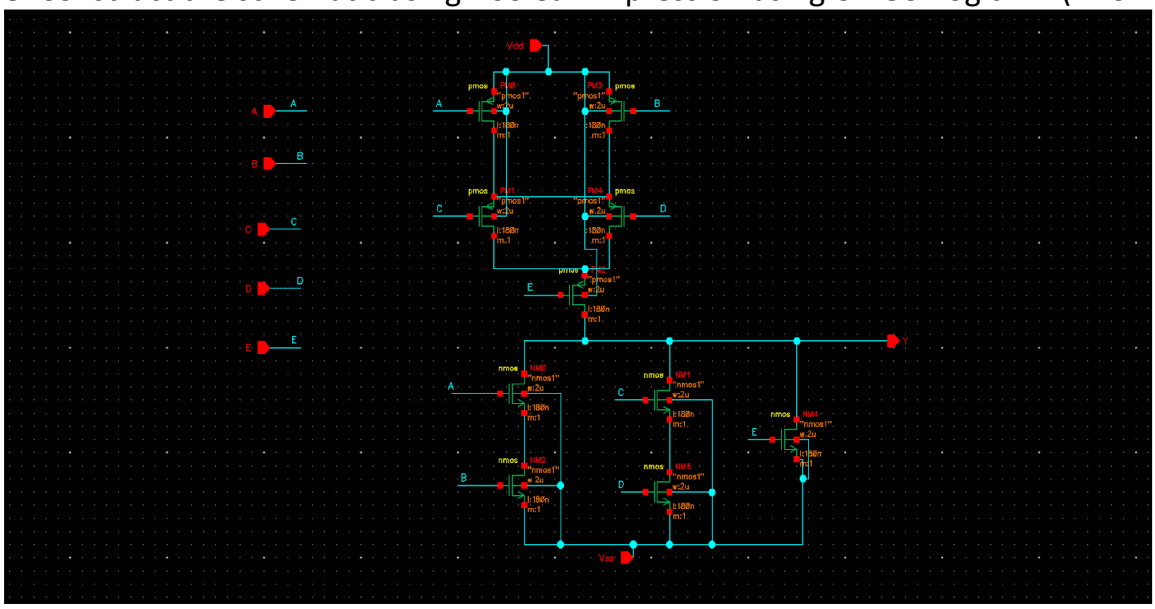

Fig.8.1: Schematic of Boolean expression using CMOS-logic

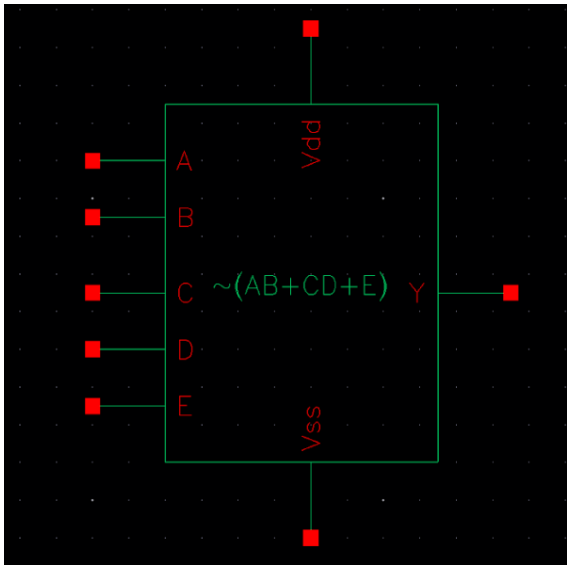

Fig.8.2: Symbol of Boolean expression using CMOS-logic

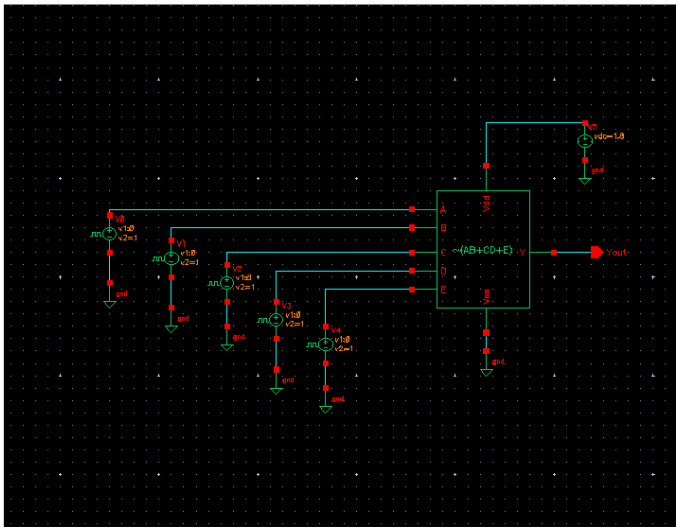

Fig.8.3: Test generation of Boolean expression using CMOS-logic

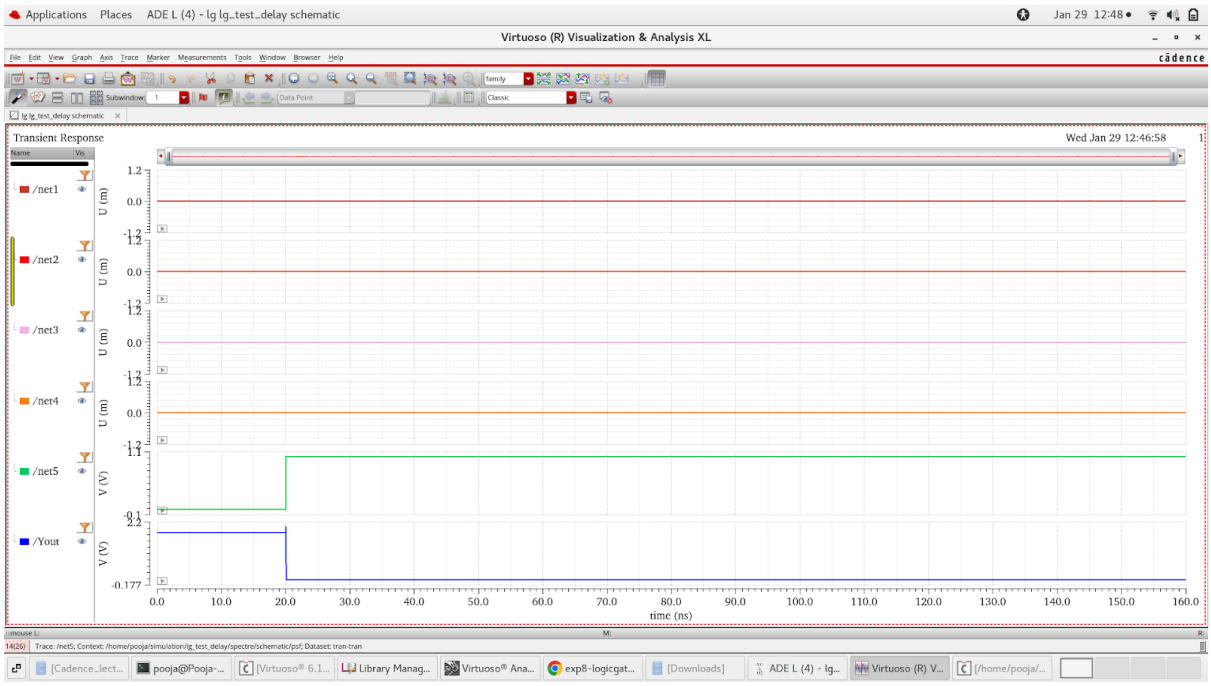

Fig.8.4: Output Waveforms of Boolean expression using CMOS-logic.

9. Common Source Amplifier With PMOS Current Mirror Load

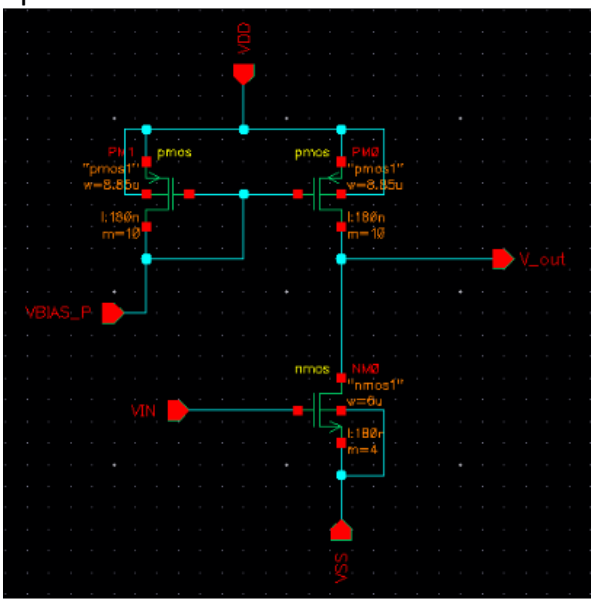

Fig.9.1: Schematic of Common Source Amplifier with PMOS Current Mirror Load

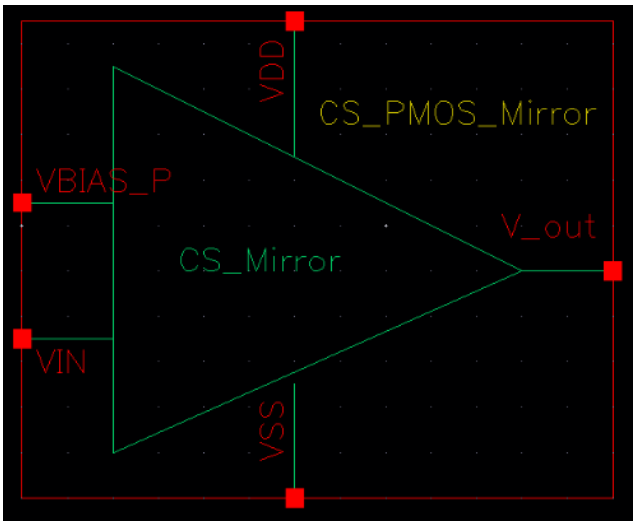

Fig.9.2: Symbol of Common Source Amplifier with PMOS Current Mirror Load

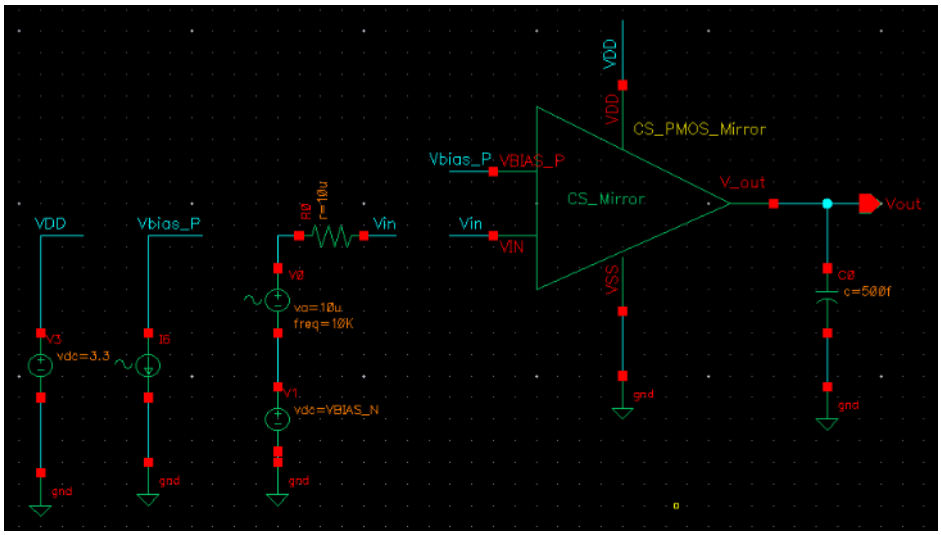

Fig.9.3: Test generation of Common Source Amplifier with PMOS Current Mirror Load

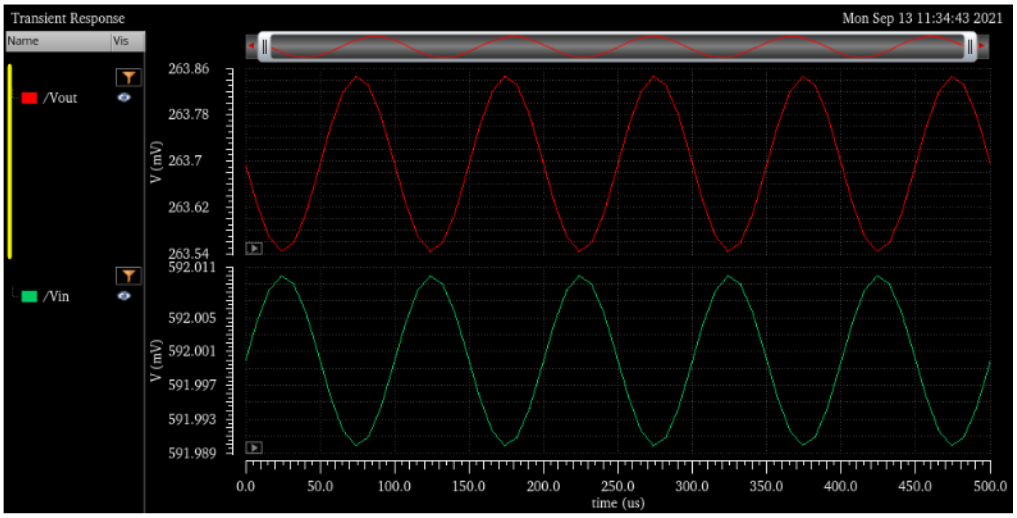

Fig.9.4: Output Waveforms of Common Source Amplifier with PMOS Current Mirror Load

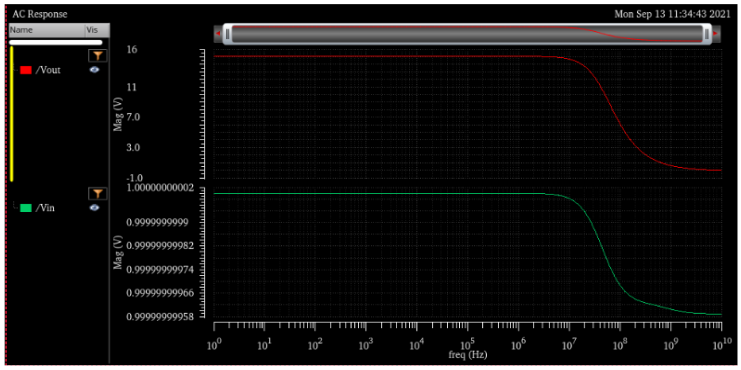

Fig.9.5: Output Waveforms - AC analysis of Common Source Amplifier with PMOS Current Mirror Load

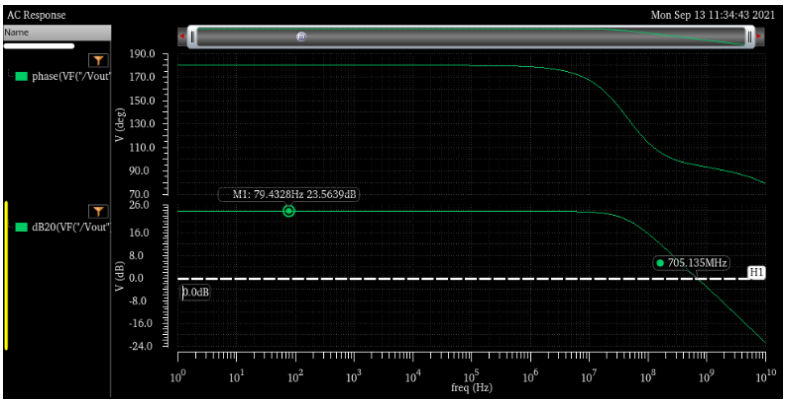

Fig.9.6: Output Waveforms – Gain and Phase plots of Common Source Amplifier

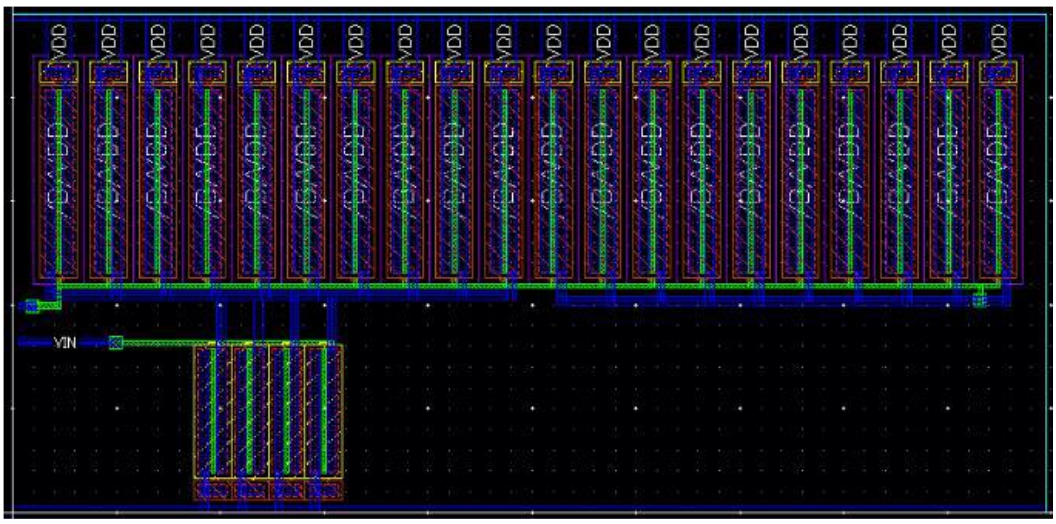

Fig.9.7: Layout of Common Source Amplifier with PMOS Current Mirror Load

09. Two Stage Operational Amplifier

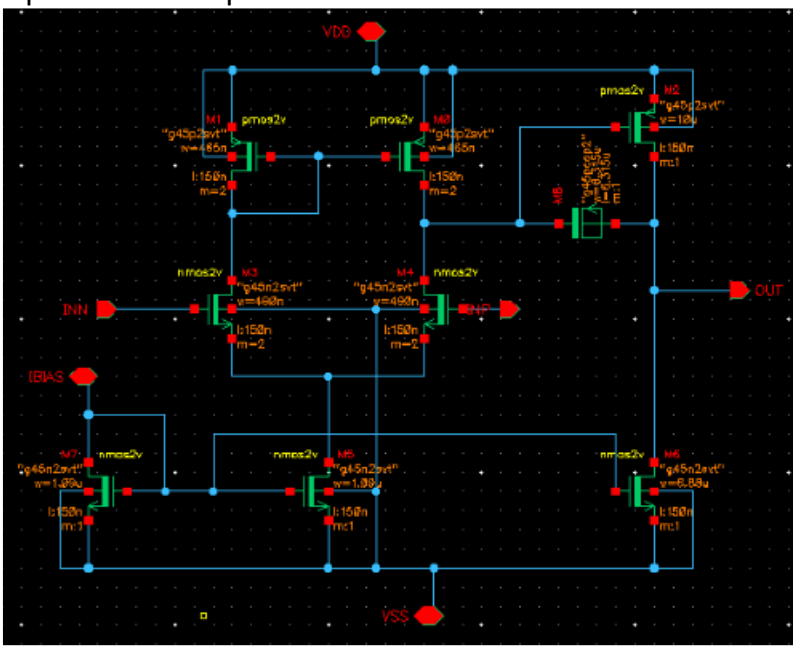

Fig.10.1: Schematic of 2 – Stage Operational Amplifier

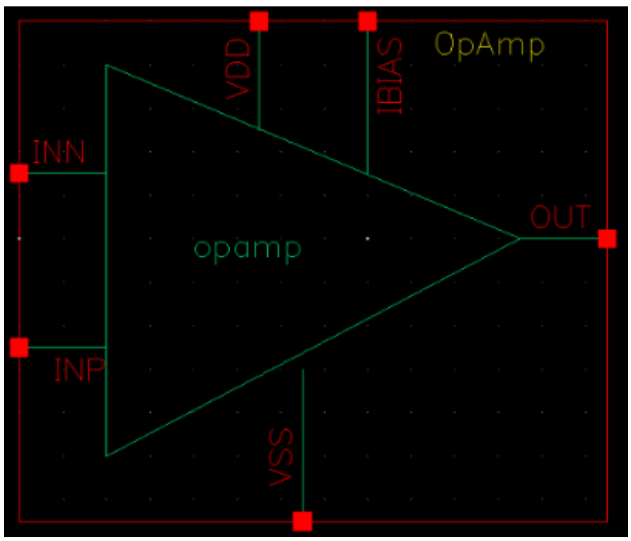

Fig.10.2: Symbol for 2 – Stage Operational Amplifier

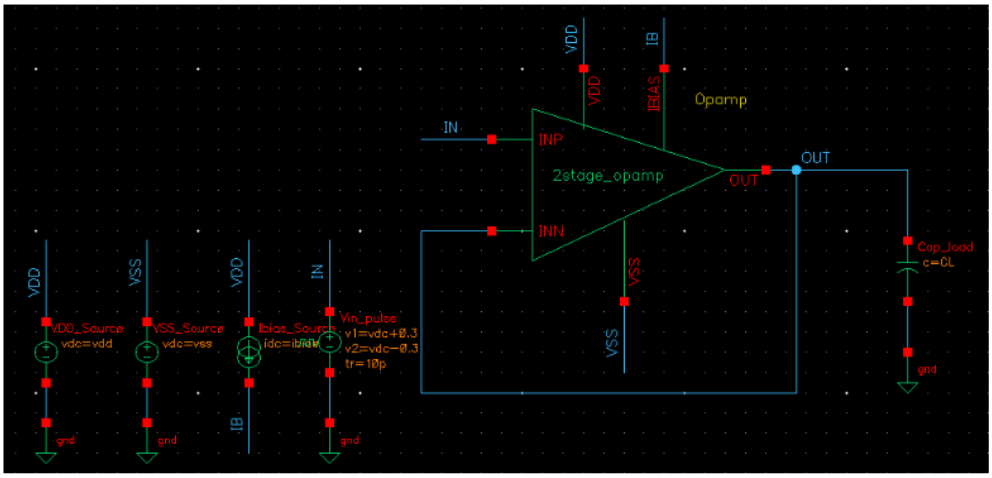

Fig.10.3: Test Schematic for 2 – Stage Operational Amplifier

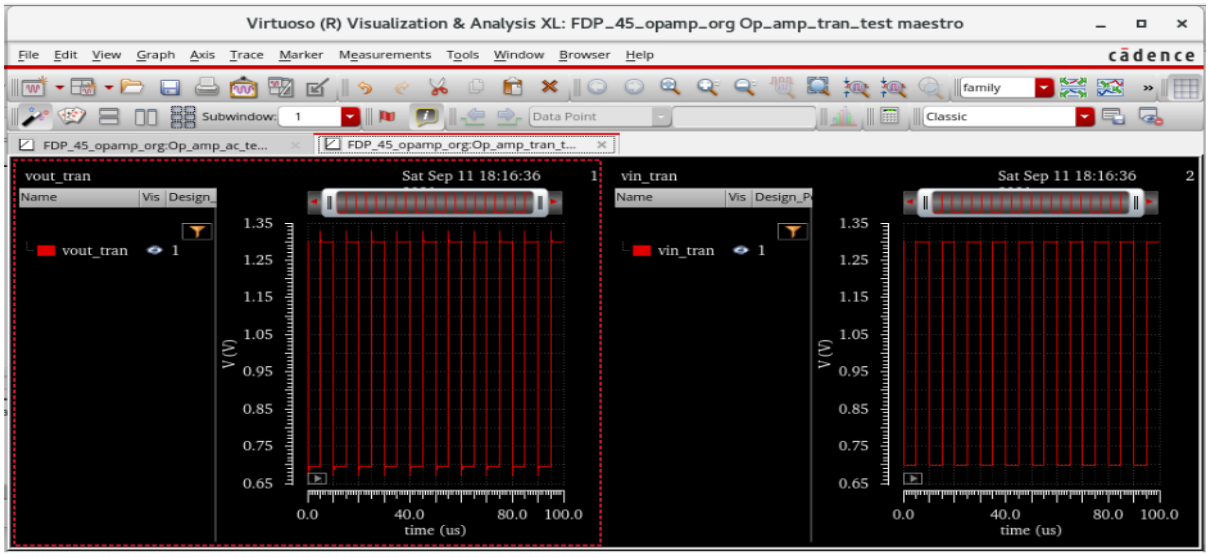

Fig.10.4: Output Waveforms

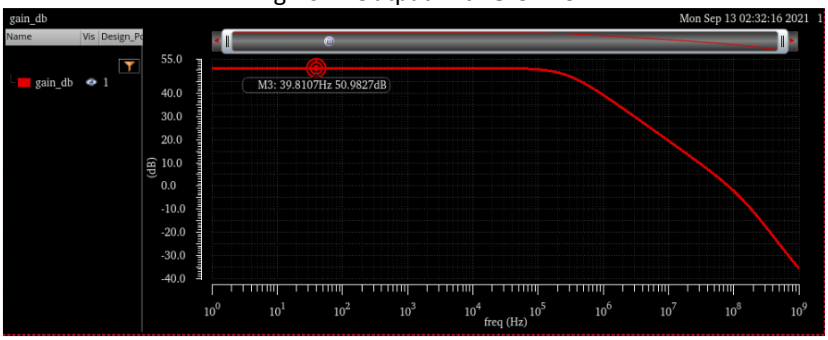

Fig.10.5: DC Open Loop Gain

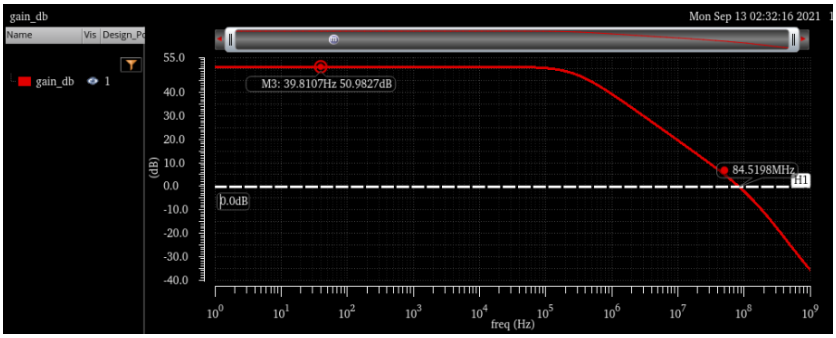

Fig.10.6: Unity Gain Bandwidth

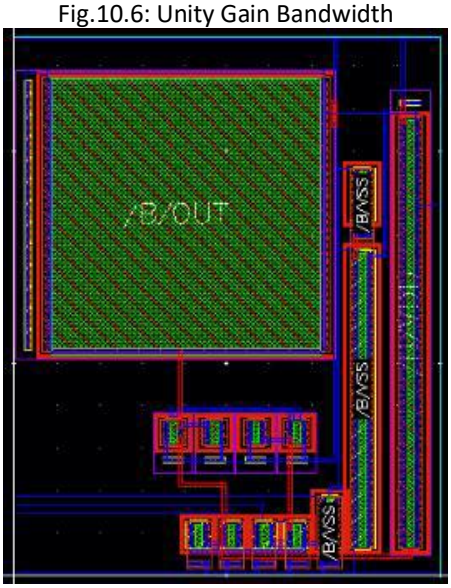

Fig.10.7: Layout for 2 – Stage Operational Amplifier

Demonstration Experiments:

11. UART- Universal Asynchronous Synchronous Receiver

uart_ROUTED_NETLIST.v

```
`timescale 1 ns / 1 ps
module uart (
    reset           ,
    txclk           ,
    ld_tx_data      ,
    tx_data         ,
    tx_enable       ,
    tx_out          ,
    tx_empty        ,
    rxclk           ,
    uld_rx_data     ,
    rx_data         ,
    rx_enable       ,
    rx_in           ,
    rx_empty
    );
// Port declarations
    input         reset          ;
    input         txclk          ;
    input         ld_tx_data     ;
    input  [7:0]  tx_data        ;
    input         tx_enable      ;
    output        tx_out         ;
    output        tx_empty       ;
    input         rxclk          ;
    input         uld_rx_data    ;
    output [7:0]  rx_data        ;
    input         rx_enable      ;
    input         rx_in          ;
    output        rx_empty       ;

// Internal Variables
    reg [7:0]     tx_reg         ;
    reg           tx_empty       ;
    reg           tx_over_run    ;
    reg [3:0]     tx_cnt         ;
    reg           tx_out         ;
    reg [7:0]     rx_reg         ;
    reg [7:0]     rx_data        ;
    reg [3:0]     rx_sample_cnt  ;
    reg [3:0]     rx_cnt         ;
    reg           rx_frame_err   ;
    reg           rx_over_run    ;
    reg           rx_empty       ;
    reg           rx_d1          ;
    reg           rx_d2          ;
    reg           rx_busy        ;

// UART RX logic
```

```
                    // Check if last rx data was not unloaded,
                    rx_over_run  <= (rx_empty) ? 0 : 1;
                  end
                end
              end
            end
          end
        end
        if ( ! rx_enable) begin
          rx_busy <= 0;
        end
      end
// UART TX Logic
      always @ (posedge txclk or posedge reset)
      if (reset) begin
        tx_reg        <= 0;
        tx_empty      <= 1;
        tx_over_run   <= 0;
        tx_out        <= 1;
        tx_cnt        <= 0;
      end else begin
         if (ld_tx_data) begin
            if ( ! tx_empty) begin
              tx_over_run <= 0;
            end else begin
              tx_reg   <= tx_data;
              tx_empty <= 0;
            end
         end
         if (tx_enable &&  ! tx_empty) begin
           tx_cnt <= tx_cnt + 1;
           if (tx_cnt == 0) begin
             tx_out <= 0;
           end
           if (tx_cnt > 0 && tx_cnt < 9) begin
              tx_out <= tx_reg[tx_cnt -1];
           end
           if (tx_cnt == 9) begin
             tx_out <= 1;
             tx_cnt <= 0;
             tx_empty <= 1;
           end
         end
         if ( ! tx_enable) begin
           tx_cnt <= 0;
         end
      end
      endmodule
```

```
// UART RX Logic
   always @ (posedge rxclk or posedge reset)
   if (reset) begin
     rx_reg        <= 0;
     rx_data       <= 0;
     rx_sample_cnt <= 0;
     rx_cnt        <= 0;
     rx_frame_err  <= 0;
     rx_over_run   <= 0;
     rx_empty      <= 1;
     rx_d1         <= 1;
     rx_d2         <= 1;
     rx_busy       <= 0;
   end else begin
// Synchronize the asynch signal
     rx_d1 <= rx_in;
     rx_d2 <= rx_d1;
// Uload the rx data
     if (uld_rx_data) begin
       rx_data  <= rx_reg;
       rx_empty <= 1;
     end
     // Receive data only when rx is enabled
     if (rx_enable) begin
       // Check if just received start of frame
       if ( ! rx_busy &&  ! rx_d2) begin
         rx_busy       <= 1;
         rx_sample_cnt <= 1;
         rx_cnt        <= 0;
       end
       // Start of frame detected, Proceed with rest of data
       if (rx_busy) begin
          rx_sample_cnt <= rx_sample_cnt + 1;
          // Logic to sample at middle of data
          if (rx_sample_cnt == 7) begin
             if ((rx_d2 == 1) && (rx_cnt == 0)) begin
               rx_busy <= 0;
             end else begin
               rx_cnt <= rx_cnt + 1;
               // Start storing the rx data
               if (rx_cnt > 0 && rx_cnt < 9) begin
                 rx_reg[rx_cnt - 1] <= rx_d2;
               end
               if (rx_cnt == 9) begin
                  rx_busy <= 0;
```

```
  //generate rxclk and txclk so that txclk is 16 times slower than rxclk
  reg [3:0] counter;
  initial begin
    rxclk=0;
    txclk=0;
    counter=0;
  end
  always @(posedge clk) begin
    counter<=counter+1;
    if (counter == 15) txclk <= ~txclk;
    rxclk<= ~rxclk;
  end

  //setup loopback
  always @ (tx_out) rx_in=tx_out;

        initial begin
                // Initialize Inputs
                reset = 1;
                ld_tx_data = 0;
                tx_data = 0;
                tx_enable = 1;
                uld_rx_data = 0;
                rx_enable = 1;
//              rx_in = 1;

                // Wait 100 ns for global reset to finish
                #500;
                reset = 0;

    // Send data using tx portion of UART and wait until data is recieved
                tx_data=8'b0111_1111;
                #500;
    wait (tx_empty==1);  //make sure data can be sent
    ld_tx_data = 1;      //load data to send
    wait (tx_empty==0);  //wait until data loaded for send
    $display("Data loaded for send");
    ld_tx_data = 0;
    wait (tx_empty==1);  //wait for flag of data to finish sending
    $display("Data sent");
    wait (rx_empty==0);  //wait for
    $display("RX Byte Ready");
                uld_rx_data = 1;
    wait (rx_empty==1);
    $display("RX Byte Unloaded: %b",rx_data);
    #100;

    $finish;
end
endmodule
```

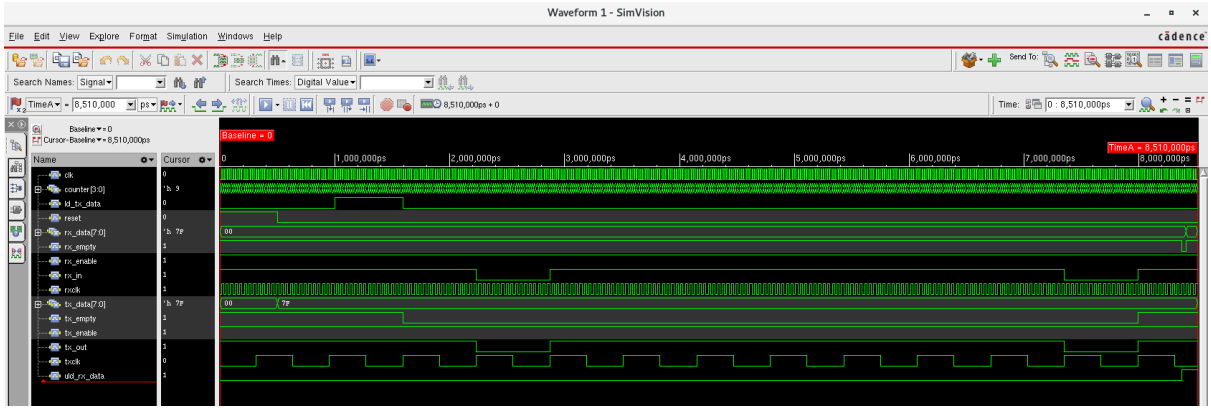

Fig.10.7: Simulation Waveform of UART

12. SRAM: Implementation of 6T-SRAM

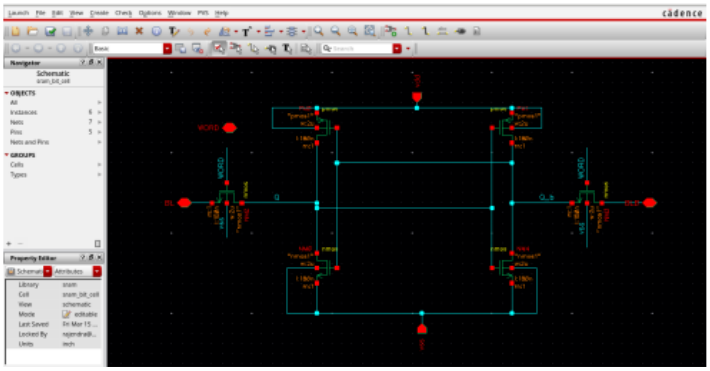

Fig.12.1: Schematic of SRAM

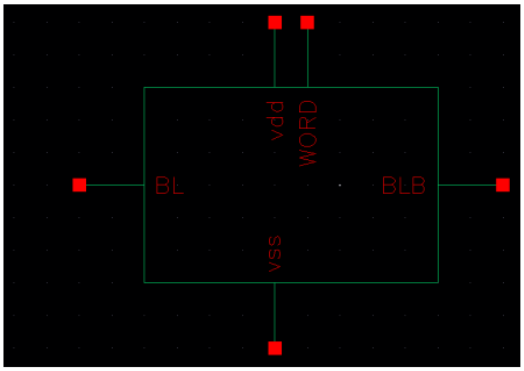

Fig.12.2: Symbol of SRAM

Fig.12.3: Output of SRAM read

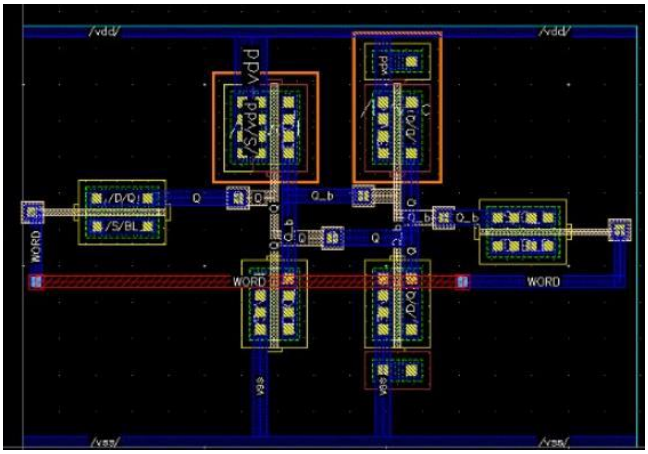

Fig.12.4: Layout of SRAM Cell.